AI ASSISTED CODING-16.2

Roll no: 2303A52454

B-34

Task 1: (Design Database Schema for Student Management System)

Prompt:

Design a relational database schema for a Student Management System using SQL. Create tables for Students, Courses, and Enrollments with proper primary keys, foreign keys, and normalized structure.

Code:

Create Students Table

```
CREATE TABLE Students (
student_id INT PRIMARY KEY,
name VARCHAR(100),    email
VARCHAR(100)
);
```

Create Courses Table

```
CREATE TABLE Courses (
course_id INT PRIMARY KEY,
course_name VARCHAR(100),
  credits INT
);
```

Create Enrollments Table (IMPORTANT ForeignKey)

```
CREATE TABLE Enrollments (
enrollment_id INT PRIMARY KEY,
student_id INT,    course_id INT,
  FOREIGN KEY (student_id) REFERENCES Students(student_id),
  FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
```

Sample Input/Output:

Create Students Table

student_id	int	NO	PRI
name	varchar(100)	YES	
email	varchar(100)	YES	

Create Courses Table

course_id	int	NO	PRI	NULL
course_name	varchar(100)	YES		NULL
credits	int	YES		NULL

Create Enrollments Table (IMPORTANT ForeignKey)

enrollment_id	int	NO	PRI	NULL
student_id	int	YES	MUL	NULL
course_id	int	YES	MUL	NULL

Explanation:

1. This task focuses on designing a normalized relational database schema.
2. The Students table stores student personal details.
3. The Courses table maintains course information and credit values.
4. The Enrollments table establishes a many-to-many relationship between students and courses.
5. Primary keys uniquely identify each record in the tables.
6. Foreign keys enforce data integrity between related tables.
7. The schema follows normalization principles to reduce redundancy.
8. This structure supports efficient data retrieval and management operations.

Task 2: (Insert Sample Records) Prompt:

Insert sample records into Students, Courses, and Enrollments tables using SQL queries. Display the inserted data to verify successful database population.

Code:

Insert into Students

```
INSERT INTO Students VALUES
(1, 'Rahul', 'rahul@gmail.com'),
(2, 'Sneha', 'sneha@gmail.com'),
(3, 'Arjun', 'arjun@gmail.com');
```

Insert into Courses

```
INSERT INTO Courses VALUES
(101, 'DBMS', 4),
(102, 'Python', 3),
(103, 'AI', 5);
```

Insert into Enrollments

```
INSERT INTO Enrollments VALUES
(1, 1, 101),
(2, 2, 102),
(3, 3, 103);
```

Sample Input/Output:

Insert into Students

student_id	name	email
abc Filter...	abc Filter...	abc Filter...
1	Rahul	rahul@gmail.com
2	Sneha	sneha@gmail.com
3	Arjun	arjun@gmail.com

Insert into Courses

course_id	course_name	credits
101	DBMS	4
102	Python	3
103	AI	5

Insert into Courses

enrollment_id	student_id	course_id
1	1	101
2	2	102
3	3	103

Explanation:

1. This task demonstrates how to populate relational tables with sample data.
2. INSERT statements are used to add student, course, and enrollment records.
3. The Enrollments table references valid student and course IDs ensuring referential integrity.
4. Proper ordering of insert operations prevents foreign key constraint errors.
5. SELECT queries help verify that the records are stored correctly.
6. Displaying table contents confirms successful database transactions.
7. This step is essential before performing query operations like joins or updates.
8. The populated database now supports real-time academic data management operations.

Task 3: (Perform CRUD Operations)

Prompt:

Write SQL queries to retrieve student details along with their enrolled course information using JOIN operations. Display filtered results based on specific conditions such as course name or credits.

Code:

Update Student Email

```
UPDATE Students
SET email = 'rahul_new@gmail.com'
WHERE student_id = 1;
```

Delete Enrollment Record

```
DELETE FROM Enrollments
WHERE enrollment_id = 3;
```

Sample Input/Output:

Update Student Email

student_id	name	email
1	Rahul	rahul_new@gmail.com
2	Sneha	sneha@gmail.com
3	Arjun	arjun@gmail.com

Delete Enrollment Record

enrollment_id	student_id	course_id
1	1	101
2	2	102

Explanation:

1. This task demonstrates execution of fundamental CRUD database operations.
2. SELECT query is used to retrieve course information from the Courses table.
3. UPDATE query modifies an existing student email ensuring accurate data maintenance.
4. DELETE query removes an unwanted enrollment record from the database.
5. WHERE clause is used to target specific rows during update and delete operations.
6. Validation using SELECT statements confirms successful modification of records.
7. These operations are essential for real-time database management systems.
8. Proper CRUD handling ensures data consistency and integrity in relational databases.

Task 4: (Execute Aggregate Queries) Prompt:

Write SQL queries using aggregate functions to analyze course and enrollment data. Calculate total number of students enrolled and average course credits.

Code:

Count Number of Students Enrolled Per Course

```
SELECT
   c.course_name,
   COUNT(e.student_id) AS student_count
FROM
   Courses c
```

```
JOIN
    Enrollments e ON c.course_id = e.course_id
GROUP BY
    c.course_name;
```

Display Courses with More Than One Student

```
SELECT
    c.course_name,
    COUNT(e.student_id) AS student_count
FROM
    Courses c
JOIN
    Enrollments e ON c.course_id = e.course_id
GROUP BY
    c.course_name
HAVING
    COUNT(e.student_id) > 1;
```

Sample Input/Output:

Count Number of Students Enrolled Per Course

course_name	student_count
abc Filter...	abc Filter...
DBMS	1
Python	1

Display Courses with More Than One Student

course_name	student_count
abc Filter...	abc Filter...

No data

<u>Explanation:</u>

Analysis of Correctness

- **The Join:** Using JOIN (Internal Join) ensures that only courses with at least one enrollment are counted. If you wanted to see courses with zero students, you would use a LEFT JOIN.
- **The Grouping:** Grouping by c.course_name is effective, though in a production environment, grouping by c.course_id is safer to avoid merging two different courses that happen to have the same name. · **The Filter:** The HAVING clause correctly targets the aggregate COUNT, ensuring the output only lists courses that meet the 'more than one' criteria.

<u>Task 5</u>: <u>(Query Optimization)</u> <u>Prompt</u>:

Create a SQL VIEW to display student names along with their enrolled course names. Use the view to retrieve simplified academic information from multiple related tables.

<u>Code</u>:

The Original Query (Subquery)

```
SELECT * FROM Students
WHERE student_id IN (
```

```
    SELECT student_id
    FROM Enrollments
    WHERE course_id = 101
);
```

The Optimized Query (JOIN)

```
SELECT s.*
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
WHERE e.course_id = 101;
```

Sample Input/Output:

student_id	name	email
abc Filter...	abc Filter...	abc Filter...
1	Rahul	rahul_new@gmail.com

Explanation:

Analysis of Correctness & Performance Correctness

- **Identical Results:** Both queries will return the exact same rows from the Students table specifically, those students who have a record in the Enrollments table for course_id = 101.
- **Duplicate Handling:** Note that if a student is enrolled in the same course multiple times (which shouldn't happen with proper constraints), the JOIN might return duplicate rows, whereas the IN subquery would not. To ensure 100% parity in all scenarios, you could add SELECT DISTINCT s.*.

Why the JOIN is "Optimized"

1. **Execution Plan:** Modern SQL optimizers are very good at handling JOIN operations. They can use "Nested Loop Joins" or "Hash Joins" to find matches much faster than evaluating a list of IDs.

2. **Indexing:** A JOIN makes it very easy for the engine to utilize an index on Enrollments.student_id and Students.student_id.
3. **Readability:** As queries grow more complex (e.g., needing data from 3 or 4 tables), JOIN syntax remains much cleaner and easier to maintain than nested layers of subqueries.